

Computing After Applications

The Machine-Native End State Under Abundant Intelligence

Saurabh Nandwana

with OpenAI Codex as research and drafting collaborator

Systems foresight preprint • Evidence cutoff: 30 August 2026

Abstract

Personal computing was organized around a human operator: files imitate paper, folders imitate cabinets, windows imitate desks, and applications bundle data, logic, and interface into products a person can navigate. Agents are now being retrofitted into that world by reading pixels, producing keystrokes, calling application-specific APIs, and occupying chat panels. This paper argues that the retrofit is transitional.

The argument is deliberately future-first. Contemporary agent benchmarks, developer-productivity studies, inference costs, and hardware bottlenecks are treated as observations of a dated model–harness–interface–organization configuration, not as ceilings. We instead reason backward from a plausible 10–30 year regime in which frontier discoveries diffuse through open weights and distillation, useful intelligence becomes dynamically routed and inexpensive, automated research loops compress software and hardware iteration, computation spans heterogeneous local and cloud fabrics, and physical actuators become increasingly callable.

Under that regime, the stable unit of computing is not the app, file, window, click, token, or model. It is the *verified semantic state transition*: a typed change to digital or physical state, proposed by a human or agent, bounded by capabilities and invariants, placed onto suitable compute or machinery, and accompanied by evidence and an outcome receipt. The resulting machine-native computer separates an authority plane, cognition plane, execution plane, and projection plane. Models and code become replaceable workers and compilation artifacts. Interfaces become generated views. Applications become capability bundles. Files become portable projections or archives. The operating system becomes a semantic transaction, learning, verification, and placement kernel.

Economically, the claim is not that all expenditure on computing vanishes. It is that reproducible implementation ceases to be a durable source of rent. Value migrates toward authoritative state, identity, energy, compute, live data, physical execution, coordination, guarantees, and liability. The paper presents the destination architecture, scenario envelope, transition path, counterarguments, and a falsifiable research program for the next few decades.

One-sentence thesis. When machine intelligence can cheaply regenerate software and interfaces, the computer’s narrow waist becomes governed state and verifiable effects; everything else—models, code, applications, files, screens, runtimes, and processors—is a replaceable implementation or projection.

1 Scope: End Game Means a Planning Horizon

This paper is not a forecast of computing centuries from now. Beyond several decades, recursive research, new physical substrates, and institutional change make detailed architecture intellectually unserious. “End game” here means the design attractor visible from 2026 for roughly 2035–2055: far enough to stop treating today’s application stack as inevitable, but close enough that claims can be tied to mechanisms, evidence, and experiments.

The destination is also not a claim of inevitability. Intelligence may remain gated, automated research may disappoint, robotics may advance unevenly, and political institutions may preserve closed stacks. The purpose is different: identify an architecture that remains useful across a wide range of capability and deployment futures, including futures substantially more capable than the systems available when this paper was written.

1.1 The error this paper corrects

An observed productivity number is generated by a complete sociotechnical system:

$$P_t = F(M_t, H_t, I_t, T_t, V_t, O_t, U_t), \quad (1)$$

where M is model capability, H the agent harness, I the machine interface, T task design, V verification, O organizational process, and U user adaptation. A study that freezes a 2025 repository, ticket format, review process,

interface, and human role measures that configuration. It does not establish the productivity of future models acting through machine-native interfaces inside reorganized firms.

METR’s early-2025 randomized study found experienced developers were 19% slower with then-current tools. Its February 2026 follow-up reported severe selection effects as developers increasingly refused AI-disallowed tasks, concurrent agents invalidated simple time accounting, and later raw estimates moved toward speedup with substantial uncertainty [1, 2]. The lesson is not to discard measurement. It is to use negative measurements diagnostically: context, harnesses, permissions, verification, repository structure, and human scheduling were bottlenecks. They are variables to redesign, not laws of nature.

The same principle applies to hardware. A shortage of a particular memory generation, the power of a current accelerator, or the cost of a current token is a launch condition. Data movement, heat rejection, latency, capital construction, and physical risk are more durable constraints; their implementations and magnitudes still move.

2 Forecasting Method: Evidence Without Snapshot Myopia

Every pivotal claim in this paper belongs to one of five classes.

Table 1: Forecast discipline used throughout the paper.

Tag	Class	Legitimate use	Misuse blocked
O	Observation	A dated result, deployment, specification, or measurement under stated conditions.	Treating present performance, cost, or adoption as a permanent ceiling.
T	Trajectory	A direction supported by repeated improvement plus a causal mechanism.	Assuming a constant exponential rate indefinitely.
C	Constraint	A physical, logical, institutional, or security property robust to better models.	Confusing today’s implementation bottleneck with the underlying constraint.
A	Scenario assumption	An explicit condition used to reason backward from a possible future.	Presenting a narrative choice as empirical fact.
D	Design implication	An architectural conclusion that should hold across several scenarios.	Claiming novelty merely because adjacent components have not yet been integrated.

Four rules follow.

1. **Benchmarks are launch coordinates, not asymptotes.** They can establish that a mechanism works, that an interface matters, or that a failure mode exists. They cannot by themselves forecast a decades-long ceiling.
2. **Capability and adoption are separate curves.** Institutions, liability, procurement, regulation, interfaces, and skills can lag a technically possible system by years.
3. **Fixed-task efficiency and aggregate demand are separate curves.** Intelligence per joule can improve rapidly while total energy use rises because agents attempt more reasoning, simulation, media, research, and physical work.
4. **The architecture should survive its own success.** Its justification cannot depend solely on models remaining unreliable, expensive, token-based, Transformer-shaped, or unable to use graphical interfaces.

2.1 Future-regime assumptions

The reference case is not infinite compute. It is *effective cognitive abundance*: for a large class of bounded digital tasks, generating, testing, and revising many machine attempts becomes cheap relative to human attention and organizational delay.

Table 2: Assumptions used to derive the destination. Their failure changes the pace more than the narrow waist.

	Assumption	Directional basis	If substantially false
A1	Useful intelligence becomes cheap and elastic.	Quality-adjusted inference cost, sparse activation, adaptive test-time compute, quantization, and specialized accelerators.	More work remains human-operated; semantic state and authority are still valuable.
A2	Frontier methods diffuse.	Open-weight releases, distillation, rapid derivative ecosystems, and small-team algorithmic invention.	The same architecture may be controlled by a few frontier providers.
A3	Agents gain longer horizons.	Better models, executable environments, memory, semantic interfaces, multi-agent decomposition, and automated verification.	Apps persist longer as guarded workflow shells.
A4	AI automates parts of AI and systems R&D.	AlphaEvolve, self-improving agent scaffolds, automated science, compilers, and chip-design loops.	Progress is steadier and easier to forecast; destination arrival slows.
A5	Hardware becomes heterogeneous and composable.	Chiptlets, 3D memory, optical links, sparse models, on-device accelerators, and open ISAs.	A smaller set of accelerator platforms remains beneath the same semantic layer.
A6	Digital agency increasingly reaches physical systems.	Vision-language-action models, robot-local inference, world models, autonomous laboratories, and fleet orchestration.	Digital implementation collapses before embodied labor; physical scarcity remains dominant.

3 Why Discontinuity Is a Serious Base Case

3.1 Discovery cost and replication cost are diverging

Frontier search may become more capital-intensive at the same time that a previously frontier capability becomes dramatically cheaper to deploy. From 2012–2023, one analysis estimated that compute required for a fixed language-model performance level halved roughly every eight months, with substantial uncertainty [3]. A later analysis of delivered capability found quality-adjusted inference becoming roughly 5–10 times cheaper annually over its measurement window, while spending at the newest frontier continued rising [4]. These rates are not laws. Their conceptual implication is stronger than their precise magnitude: hardware-only extrapolation misses algorithmic and systems progress.

DeepSeek-R1 provided an early concrete example of expensive reasoning discovery transferred into open distilled students from 1.5B to 70B parameters [5]. Sparse systems make total parameter count increasingly different from active computation; Qwen3 and later open families activate a fraction of their stored capacity [6]. Adaptive test-time computation further converts intelligence into a task-specific budget rather than a fixed model property [7].

T The likely substrate is therefore not one monolithic model. It is a dynamically assembled cognitive working set:

$$\mathcal{C}_\tau = \{B, E_\tau, M_\tau, D_\tau, A_\tau, W_\tau, V_\tau\}, \quad (2)$$

where a base model B is combined with selected experts E , memory M , reasoning depth D , tools and agents A , mutable task-local learning W , and verifiers V . A future operating system schedules this set under latency, privacy, energy, authority, and risk constraints.

3.2 Thought itself need not remain token-shaped

Coconut recycles continuous hidden states as reasoning state rather than translating every step into text [8]. Large Concept Models operate autoregressively in sentence-level embedding space [9]. Interlat explores fully

latent communication among agents, while StateBridge investigates training-free alignment between heterogeneous hidden-state spaces [10, 11]. Diffusion language models and recurrent memory architectures provide other departures from strict next-token execution [12, 13].

These are early systems, not proof of one successor architecture. They establish a narrower proposition: natural-language chain-of-thought, token billing, static weights, and one-token-at-a-time decoding are contingent design choices. A machine-native computer must tolerate their disappearance.

C This makes internal interpretability an unsafe authority mechanism. Even a model that verbalizes reasoning may omit the causal basis of its action. Trust must rest on bounded capabilities, independent checks, state diffs, simulations, tests, and observable outcomes—not on accepting a generated explanation as a proof.

3.3 Research begins to enter a feedback loop

AlphaEvolve combined model-generated programs with automated evaluation and improved deployed scheduling, low-level kernels, and accelerator logic; some improvements benefited the computing infrastructure used by AI itself [14]. Darwin Gödel Machine modified its own coding-agent scaffold and empirically retained better descendants [15]. AI Scientist v2, Co-Scientist, and autonomous laboratory work demonstrate partial loops from hypothesis through experiment or evaluation [16, 17, 18].

This is not evidence of unconstrained takeoff. It is evidence that the human research loop is no longer indivisible. Hypothesis generation, implementation, simulation, evaluation, literature synthesis, and experimental planning can be multiplied without multiplying human headcount. The remaining scarcity moves toward good evaluators, physical experiments, trusted data, frontier-scale integration, and judgment over objectives.

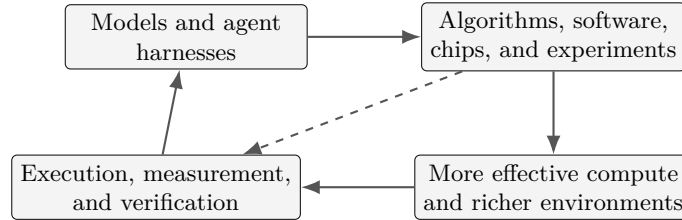


Figure 1: A bounded recursive efficiency loop. Each edge can improve before end-to-end autonomous self-improvement exists.

4 The Machine-Native Computer

4.1 The architectural claim

The personal computer’s authoritative objects were optimized for human manipulation: windows, files, application processes, menus, and pixels. The machine-native computer instead treats humans, agents, services, sensors, and robots as actors over shared state.

Let a computer at time t be

$$\mathcal{M}_t = \langle S_t, A_t, K_t, C_t, X_t, P_t, L_t \rangle, \quad (3)$$

where:

- S_t is federated canonical semantic state;
- A_t is the set of human, agent, organizational, and device principals;
- K_t is the deterministic authority and commit kernel;
- C_t is the elastic cognition substrate;
- X_t is the digital and physical execution fabric;
- P_t is the set of generated and legacy projections;

- L_t is the evidence, learning, and provenance lifecycle.

The narrow waist is not a universal ontology containing every concept. It is a small, stable contract for identifying state, proposing changes, granting authority, stating dependencies and invariants, classifying reversibility, executing through adapters, and producing receipts. Domain schemas can evolve above it.

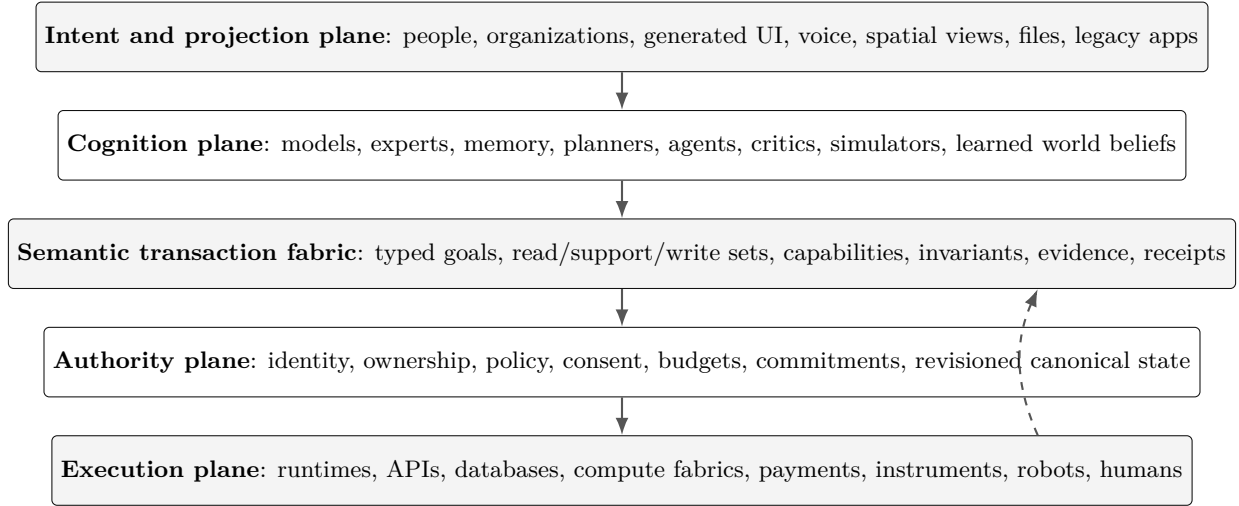


Figure 2: Destination architecture. Probabilistic cognition proposes; deterministic authority commits; execution returns evidence; interfaces are projections.

4.2 Canonical state is not a giant database

Canonical means authoritative for a scope, not centralized or omniscient. The state fabric is a federation of typed, versioned objects with stable references, ownership, provenance, schema versions, and explicit authority. It can include local-first replicas, organizational systems, public registries, and physical-world beliefs.

A crucial distinction is:

- **authoritative commitment state:** balances, ownership, contracts, identity, permissions, accepted designs, scheduled obligations, and committed project revisions;
- **probabilistic world belief:** what sensors or models infer to be present, reachable, damaged, risky, or likely, with confidence, freshness, source, and observation time.

World models may become extraordinarily capable, but a plausible latent belief is not the same object as a bank balance, consent grant, or signed contract. The system must join inference to commitment without confusing them.

4.3 The model must not be the kernel

This separation is not justified only by present model fallibility. A superhuman administrator still should not possess unlimited, ambient authority. The cognition plane may infer goals, generate plans, learn preferences, write code, and propose changes. A smaller deterministic kernel validates capabilities, current revisions, declared invariants, risk budgets, and required evidence before effects occur.

WASI’s no-ambient-authority model, seL4’s capability microkernel, and CHERI’s hardware-software memory capabilities provide lower-level precedents [19, 20, 21]. A machine-native kernel extends the question from whether a component may open a socket to whether a delegated actor may expose specific fields, spend an amount, update a class of state, or actuate machinery inside an envelope.

5 The Fundamental Unit: A Semantic Transaction

A proposed change is represented as:

$$\tau = \langle p, g, b, R, U, W, \kappa, \Phi, \Delta, \nu, q, \rho \rangle, \quad (4)$$

where p is the principal, g the typed goal, b the base revision, R the observed read set, U the *support set* that must remain true, W the intended write set, κ the delegated capabilities, Φ the operations and preconditions, Δ the predicted semantic and external effects, ν the verification plan, q the risk and reversibility class, and ρ the inverse or compensating plan.

The distinction between R and U is important. An agent may read a large amount of context while relying on only a subset for correctness. Rejecting every transaction whenever any observed object changes creates needless serialization. Ignoring dependencies creates stale plans. Support-aware concurrency asks what facts the action actually depends upon.

A transaction can commit only if:

$$\begin{aligned} \text{Commit}(\tau, S_t) \iff & \text{Authorized}(\kappa, p, \tau) \wedge \text{SupportTrue}(U, S_t) \\ & \wedge \text{InvariantsHold}(\Phi, \Delta, S_t) \wedge \text{RiskWithinBudget}(q) \\ & \wedge \text{EvidenceSufficient}(\nu). \end{aligned} \quad (5)$$

The result is not merely success or failure. It is a new state revision plus a receipt binding the delegated intent, prior state, proposed change, policy versions, executor, model and tool versions where relevant, evidence, external acknowledgements, cost, resulting state, and compensation pointer.

5.1 Concurrency should be semantic

Multiple human and machine actors will operate simultaneously. The kernel can classify operations:

- **monotone or commutative:** merge without coordination;
- **invariant-confluent:** execute independently while declared invariants remain true;
- **invariant-sensitive:** coordinate, lock, or obtain a lease;
- **digitally reversible:** record an inverse commit;
- **compensatable:** use a saga when an external effect cannot be rolled back;
- **irreversible or safety-critical:** stage, simulate, require stronger proof or consent, and monitor postconditions.

CRDTs and local-first systems solve mechanical convergence for appropriate data types; invariant confluence and CALM characterize when coordination can be avoided; sagas model long-lived external operations [22, 23, 24, 25]. None alone provides cross-domain intent, authority, evidence, and physical-effect semantics. The transaction fabric integrates those concerns.

5.2 The success metric changes

Tokens per second, lines of code, and tasks per benchmark are local metrics. The machine-native objective is closer to validated consequential progress per scarce resource:

$$\pi^* = \arg \max_{\pi} \mathbb{E}[V(G, S_{t+1}) - L(\text{failure})] - \lambda_E E - \lambda_T T - \lambda_S C - \lambda_H H - \lambda_R R, \quad (6)$$

where H is human attention and R is risk. As cognitive attempts become cheap, verification, attention, latency, energy, and physical consequences dominate scheduling decisions.

6 Why Better GUI Agents Do Not Remove the Need

Semantic interfaces already demonstrate that the interface changes effective intelligence. ASIL exposed structured state and semantic actions across 15 applications; on its reported evaluation, frontier models improved from low GUI success to above 80% through the semantic layer, while a smaller open model improved further with training [26]. CI4A similarly exposed component state and executable operations, outperforming pixel-centric interaction on its web benchmark [27]. These are dated results, but their forward meaning is structural: the same model becomes more capable when hidden state and meaningful operations are directly addressable.

Suppose, however, that future multimodal models make cursor movement essentially free. The machine-native layer still matters for five reasons:

1. A screen is an incomplete projection; hidden state, ownership, policy, provenance, and confidence may not be visible.
2. Pixels do not define which concurrent changes commute or which invariant was relied upon.
3. A generated click trace does not itself prove delegated authority.
4. Irreversible external effects need preconditions, staged commitment, and compensation.
5. Stable state and receipts must survive replacement of the model, interface, and application.

Therefore the human-interface tax is not the foundation of the thesis. It is merely an early symptom. Even a perfect interface user benefits from a shared transactional substrate.

7 Apps, Files, and Code Become Projections

Applications bundle five things because human software production and distribution made bundling economical: a private state model, business logic, user interface, permission boundary, and commercial package. When implementation and presentation can be generated around shared state, the bundle decomposes.

Table 3: Decanonization of the human-computing stack. Existing forms may persist as useful projections.

Current object	Machine-native successor	What remains valuable
Application	Versioned capability pack: schemas, operations, validators, migrations, policies, and projections	Domain semantics, live connections, reputation, guarantees
File	Portable snapshot, content-addressed bundle, publication, interchange view, or archive	Ownership, transfer, long-term readability, air-gapped use
Folder	Query, collection, namespace, or policy boundary	Human organization and access control
Window	Temporary lens generated for a task, person, device, and risk level	Visual comparison, monitoring, precise manipulation, taste
Process	Durable goal, actor commitment, workflow, transaction, or policy	Resource and authority lifecycle
Save	Validated semantic commit	Deliberate commitment and provenance
Undo	Inverse commit or compensating workflow	Recovery from digital and external effects
Permission prompt	Time-, purpose-, object-, and budget-bounded capability lease	Consent and legitimate authority
App store	Registry of capabilities, schemas, validators, models, projections, and trust evidence	Discovery, certification, reputation
Source code	Regenerable implementation artifact derived from specifications, tests, constraints, and state	Reproducibility, inspectability, optimization, fallback

7.1 Code becomes more like a binary than a manuscript

In the reference regime, handwritten code is neither absent nor forbidden. It loses primacy. Agents may synthesize an implementation, prove or test it against a specification, deploy it inside a capability sandbox, measure it, and replace it hours later. Durable assets move toward:

- semantic specifications and state;
- executable tests, evaluators, and safety envelopes;
- policies and delegation graphs;
- provenance and outcome histories;
- training environments and domain knowledge;
- hardware mappings and empirical performance evidence.

The strongest openness requirement is consequently not a public repository. It is the ability to inspect and migrate state, understand semantics, reproduce evaluations, substitute models and executors, audit authority, and rebuild implementation without a vendor’s permission.

7.2 Interfaces are compiled, not installed

A projection compiler generates:

$$P = f(S, g, u, d, r, a), \quad (7)$$

from semantic state S , goal g , user u , device d , risk r , and accessibility needs a . The result may be a dense visual workspace, a voice exchange, a spatial model, code, a single approval card, or no foreground interface.

A2UI and generative-UI research are early protocols for safe declarative rendering [28, 29]. Screens do not disappear: visual-spatial inspection, monitoring, comparison, teaching, and aesthetic judgment remain high-bandwidth human activities. What disappears is the assumption that the screen is authoritative state or the only control plane.

8 The Future Operating System Schedules Cognition

Current agent operating systems schedule model calls, context, memory, and tools [30]. That is useful but incomplete. A machine-native OS additionally performs:

- semantic compilation from goals to proposed transactions;
- dynamic selection of models, experts, memories, reasoning depth, tools, and verifiers;
- memory-locality and expert-prefetch planning;
- capability issuance, information-flow enforcement, and attestation;
- support-aware concurrency and state reconciliation;
- energy, latency, cost, privacy, and thermal placement;
- schema evolution and projection generation;
- postcondition monitoring, evidence capture, and learning.

8.1 Models become pageable capability spaces

Sparse experts, adapters, retrieval, conditional memory, and personal learning suggest an analogy to virtual memory:

- total learned capacity is an address space;
- active experts and memories are a working set;
- on-chip SRAM and HBM are fast intelligence caches;

- DRAM, storage, and remote nodes hold colder capacity;
- predictive routing prefetches likely experts and state;
- the scheduler trades quality, movement, latency, privacy, and energy.

The relevant performance quantity becomes verified utility per byte moved and active operation, not parameter count. This is especially important if model architectures evolve toward latent recurrence, non-autoregressive revision, conditional lookup memory, and online adaptation.

8.2 Learning becomes a governed lifecycle

If deployed systems update personal memory, fast weights, or domain experts, training is no longer a one-time manufacturing event. State tiers may include global foundation weights, downloadable experts, organization memory, personal long-term memory, session adaptation, current activations, and external canonical state.

The OS must govern:

- which experiences may update which tier;
- provenance of learning updates;
- poisoning isolation and information-flow control;
- checkpoint, rollback, consolidation, and forgetting;
- ownership and export of personal learned state;
- the boundary between private experience and shared improvement.

Model-collapse research cautions that recursive imitation without new information can erase rare modes [31]. Sustainable synthetic learning remains grounded through execution, proof, measurement, adversarial competition, independent evaluation, human observation, or physical contact.

9 Hardware After the GPU Monoculture

9.1 Fixed-task abundance and aggregate scarcity coexist

The IEA’s April 2026 analysis reported energy per fixed AI task falling by at least an order of magnitude annually in recent years, while richer reasoning, video, and agentic queries consumed orders of magnitude more energy and aggregate AI-focused data-center electricity rose sharply [32]. The precise recent rate cannot continue forever. The important result is that efficient tasks and expanding total demand are not opposing forecasts.

C Thermodynamics, heat rejection, data-movement distance, speed-of-light latency, fabrication, grid construction, and physical reliability remain real. **T** The amount of useful cognition obtained before those constraints bind can still change by orders of magnitude through architecture and co-design.

9.2 A heterogeneous capability fabric

Current launch signals already point away from one accelerator for every phase. Google split its eighth TPU generation around training and agentic inference requirements; new systems allocate large memory and interconnect specifically to reasoning state and sparse expert traffic [33]. HBM4, CXL memory pooling, chiplets, 3D integration, optical interconnect, and wafer-scale packaging move old limits rather than accepting them [34, 35, 36].

Other substrates enter selectively:

- near- and in-memory computing reduce movement for tolerant kernels [37];
- photonics first changes rack, package, and chiplet communication, with compute likely remaining hybrid;
- event-driven sensors emit sparse state changes rather than human-oriented frame streams;

- neuromorphic and analog components serve low-power perception or stable inference niches;
- digital, exact, capability-secure control remains around approximate accelerators.

The future machine exposes requirements rather than hard-coding a device:

latency deadline; precision and error tolerance; privacy and locality; monetary and energy budget; determinism; attestation; physical-risk class; reversibility; and evidence retention.

The scheduler maps these requirements across CPU, GPU, NPU, sparse or decode engines, memory tiers, local devices, data centers, simulators, and physical actuators.

9.3 Open hardware: design abundance, fabrication scarcity

RISC-V provides an open ISA; UCIE defines open chiplet interconnect; OpenROAD automates open RTL-to-GDSII flows; OpenTitan has moved an open silicon root of trust into commercial production [38, 35, 39, 40]. These are strong leading indicators for open and inspectable instruction sets, RTL blocks, security roots, compiler stacks, and reference systems.

They do not make leading-edge fabrication free. Fabs, masks, process recipes, advanced packaging, HBM, yield learning, power, water, materials, and geopolitical coordination remain capital-intensive. Agents can collapse design labor and explore more architectures. They cannot generate a modern fab from source code. A plausible equilibrium is open designs executed by expensive foundry utilities.

10 From Digital Work to Physical Consequence

The boundary “software that causes real movement” is directionally right but too narrow. A digital action can be consequential without moving matter: transferring money, granting access, filing a tax return, publishing an authoritative record, signing a contract, modifying a medical order, or changing legal ownership. The common property is not physicality. It is an authorized state transition whose consequences escape a private simulation.

10.1 Embodied agents are capability endpoints

Gemini Robotics, Physical Intelligence’s generalist policies, NVIDIA Cosmos, and robot-local accelerators demonstrate the emerging stack: high-level embodied reasoning, world prediction or simulation, and local action policies connected to specific bodies [41, 42, 43, 44]. Current real-world reliability remains far below the destination. That observation determines deployment order, not the eventual interface.

A physical transaction extends Equation 4 with:

- a belief state with uncertainty and freshness;
- an embodiment and calibrated safety envelope;
- simulation or digital-twin evidence where useful;
- local real-time controllers that cannot wait for a cloud round trip;
- staged commitment for irreversible operations;
- post-action sensing and reconciliation;
- compensation rather than fictional rollback.

The high-level agent may request that component X be placed into assembly Y. A local controller enforces collision, force, stability, and emergency-stop constraints. The authority kernel verifies scope and preconditions. Sensors update belief. The state fabric records what was intended, attempted, observed, and accepted.

10.2 Near-infinite research means human-independent iteration, not infinite physics

In digital domains, hypotheses, implementations, tests, and simulations can multiply rapidly. Autonomous laboratories add instruments and robots to close physical loops [18, 45]. Biology, materials, energy, and robotics still encounter experiment duration, contamination, supply chains, safety, and measurement error. The system’s role is to keep those physical constraints visible as schedulable resources and evidence, rather than hiding them behind an application dashboard.

11 The Economics: The End of Implementation Rent

The claim that paid software disappears is strongest when stated precisely:

Code becomes abundant; paid access to generic software functionality collapses. Reproducible implementation ceases to be a durable source of rent.

The price of a digital service can be decomposed as:

$$P = C_{\text{compute}} + C_{\text{operations}} + R_{\text{state}} + R_{\text{authority}} + R_{\text{network}} + R_{\text{distribution}} + R_{\text{verification}} + R_{\text{liability}} + R_{\text{IP}} + R_{\text{implementation}}. \quad (8)$$

The forecast is not that every term tends to zero. It is that $R_{\text{implementation}}$, rent earned merely from possessing reproducible code, is compressed aggressively.

11.1 The unit of sale migrates

The likely sequence is:

$$\text{pay for tool access} \longrightarrow \text{pay for machine work} \longrightarrow \text{pay for verified consequential change}. \quad (9)$$

Outcome- and action-based pricing in support, sales, and enterprise-agent platforms is an early commercial signal, not proof of the final market [46, 47]. Seat pricing becomes unstable when one human supervises many simultaneous workers and when an agent, rather than a licensed person, performs most interactions.

Table 4: Exposure to implementation abundance. Durability comes from institutional or physical position, not unusually difficult code.

High exposure	Transitional / medium exposure	Structurally more durable
Generic CRUD and dashboards; thin API wrappers; basic reporting; template media; simple internal workflow tools; commodity website builders	Collaborative creative systems; complex engineering environments; analytics and planning; security tooling; domain workflows with important tacit knowledge	Identity and access authority; payments and settlement; official systems of record; regulated execution; networks and marketplaces; physical logistics; guarantees, insurance, and liability-bearing services

The durable categories do not survive because their implementation cannot be regenerated. They possess authoritative state, legal recognition, current data, distribution, network participation, risk capital, physical assets, or the right to produce consequences.

11.2 Incumbent shock and incumbent defense

Incumbents face feature-rent collapse, seat compression, interface disintermediation, suite unbundling, machine-selected distribution, and a legacy architecture tax. Yet the transition can initially strengthen them because they already control customer state, permission graphs, contractual relationships, and regulated integrations.

The decisive policy variables are state portability, write-capable interoperability, model substitutability, independent verification, user-owned identity and memory, and liability rules. Without them, apps may disappear only to be replaced by an agent oligopoly.

11.3 Patents and copyright do not simply vanish

Machine generation weakens code authorship as a moat and expands the volume of independent implementation and prior art. It does not automatically defeat patents, licenses, trade secrets, trademarks, or access controls. Independent generation is not a patent defense. Large portfolios may become a stronger friction mechanism against small agent-generated competitors unless law changes.

The likely shift is from implementation exclusivity toward data rights, identity, contractual access, certification, brand, and litigation leverage. An open future is therefore a political and standards outcome, not an automatic consequence of cheap generation.

12 Human–Agent Collaboration After the Keyboard

The keyboard, mouse, touch interface, and screen were solutions to a human motor and perception problem. They need not be the agent’s internal language. But human agency should not be replaced with a model’s inferred preferences.

Human roles move toward:

- selecting ends, values, constraints, and legitimate principals;
- resolving genuine ambiguity and conflicts among stakeholders;
- expressing taste and meaning;
- granting consent for irreversible or rights-affecting changes;
- auditing exceptions, distributions, and long-run consequences;
- governing which objectives may be optimized at all.

Agents handle search, implementation, coordination, monitoring, simulation, negotiation within bounds, and routine execution. A person may interact through speech, sketch, spatial manipulation, direct editing, a generated decision surface, or no active interface. The same transaction path serves both human gestures and agent proposals; neither receives a hidden bypass.

C Human attention can become economically scarce without becoming morally optional. Prediction of a preference is not identical to legitimate consent. Machine-native identity and delegation must preserve the distinction.

13 Transition: Build Backward From the Destination

The clean-slate model is a destination architecture, not a requirement to discard every legacy system on day one. GUI automation, MCP, WebMCP, A2A, declarative UI, APIs, and application scripting are migration bridges [48, 49, 50]. They make old state reachable and reveal which semantic capabilities should become native.

Table 5: Indicative horizons from 2026. These are capability stages, not date promises.

Stage	Approximate horizon	Dominant form	Architectural milestone
Semantic retrofit	0–5 years	Agents operate apps through APIs, semantic interfaces, and GUI fallback	Agent identities, capability leases, typed observations/actions, receipts in high-consequence verticals
State mirroring	3–8 years	Application state is synchronized into user- or organization-controlled semantic graphs	Shared IDs, provenance, schema lenses, model-neutral transactions, generated projections
Application decomposition	5–15 years	Apps separate into capability packs, validators, policies, and views	Direct human manipulation and agent action compile to one transaction IR
Machine-native production	10–25 years	Most ordinary digital work is agent-to-agent over shared state	Code and UI are routinely regenerated; cognition and execution are dynamically placed
Ambient and embodied fabric	15–30 years	Sensors, robots, labs, vehicles, and infrastructure become callable capabilities	Symbolic commitments and probabilistic world beliefs reconcile through safety-bounded physical transactions

13.1 Scenario envelope

Table 6: The architecture is designed to remain useful under sharply different futures.

Scenario	Defining assumptions	Shape of software	Scarce / paid layer
Harness-limited continuity	Reliability and institutional adaptation improve slowly	Conventional apps persist behind agents and semantic adapters	Seats, support, integration, compliance
Cognitive abundance	Capability, cost, autonomy, and verification compound	Generic app logic and UI are generated; state fabrics dominate	Compute, state, authority, outcomes, guarantees
Open sovereign substrate	Open systems stay near frontier; portability standards win	Public capability commons with private local projections	Hardware, hosting, regulated connectors, physical execution
Centralized agent oligopoly	Compute, defaults, identity, and liability concentrate	Apps dissolve into a few agent/cloud platforms	Platform usage, identity, storage, bundled frontier intelligence
Recursive acceleration	Agents materially accelerate model and hardware R&D	Software becomes continuously synthesized policy	Energy, compute reservations, control rights, verification, physical resources

The most defensible central case is cognitive abundance with uneven automated R&D, mixed with a political contest between sovereign openness and centralized platforms. The narrow waist—state, authority, transactions, evidence, and placement—matters in every row.

14 Failure Modes and Counterarguments

14.1 The universal schema fails

A single ontology can ossify, centralize power, or collapse under domain diversity. The answer is a minimal transaction core plus federated schemas, stable identifiers, versioned ontologies, and bidirectional migrations. Cambria-like lenses are relevant because schema evolution is normal, not exceptional [51].

14.2 Verification remains more expensive than generation

This is plausible. Cheap code may initially create more review work and attack surface. The architecture does not assume verification is free; it makes evidence and risk explicit, allocates stronger verifiers selectively, and treats human attention as a budget. If automated verification improves, the same layer scales rather than becoming obsolete.

14.3 End-to-end latent systems make symbolic state unnecessary

Latent cognition may dominate internal reasoning and communication. External commitments still require shared reference, concurrency, rights, and observable consequences among actors that do not share weights or objectives. The boundary can be symbolic and typed even when the computation behind it is latent.

14.4 Open systems lose to trusted closed systems

Open weights do not erase switching costs, defaults, contracts, uptime, or liability. A closed provider can remain economically dominant even with an adequate open substitute. The open outcome requires portable state and identity, interoperable capabilities, reproducible evidence, usable operations, and adequate hardware—not only source availability.

14.5 Jevons effects expand total spending

This paper predicts compression of implementation rent, not necessarily a smaller computing economy. A hundredfold fall in cost can create more than a hundredfold increase in demand. Spending can migrate toward compute, energy, robotics, data, verification, and consequences while software seats disappear.

14.6 Robotics stays narrow

If actuation, batteries, reliability, maintenance, and liability improve slowly, digital abundance will arrive much earlier than physical labor abundance. The architecture degrades gracefully: physical capabilities remain scarce external executors with stricter evidence and authority.

14.7 Security and surveillance dominate

A canonical state fabric can become an unprecedented instrument of control. Generated code can expand supply-chain risk; personalized learning can leak intimate state; agent delegation can obscure responsibility. User ownership, local execution, compartmentalization, minimal disclosure, purpose-bound capability leases, independent receipts, and plural implementations are therefore constitutional requirements, not optional product features.

15 Falsifiable Research Program

The paper is useful only if it creates evidence that can prove it wrong.

15.1 Core experiments

1. **Interface scaling.** Compare screenshot/click, semantic observation/action, and full transaction interfaces across small open, local, and frontier models. Measure outcome, steps, bytes/tokens, energy, training samples, recovery, and severity of unauthorized effects. Prediction: semantic gains persist or grow with capability.
2. **Shared-state prototype.** Place documents, tables, calendar objects, communication, and media timelines over one state/transaction layer. Require two independent agents and a human to work concurrently through both generated UI and language.
3. **Model-swap ABI.** Replace model family, scale, quantization, and runtime without migrating canonical state or weakening transaction semantics. Prediction: cognition becomes substitutable behind a stable effect boundary.
4. **Capability-kernel red team.** Compare a god-mode GUI agent with proposal/commit execution under prompt injection, confused-deputy attacks, malicious tools, exfiltration, budget abuse, and stale state.
5. **Semantic concurrency.** Compare serialization, CRDT-only merging, and support/invariant-aware execution on multi-agent workflows. Measure conflicts, correctness, latency, and unnecessary replanning.
6. **Receipt replay.** Test whether an independent party can reconstruct what was authorized, attempted, executed, and achieved without trusting the model provider.
7. **Generated projection study.** Back fixed and generated interfaces with identical state. Measure completion, cognitive load, accessibility, error discovery, and inappropriate automation.
8. **Physical saga simulation.** Join a contract, payment, inventory update, and robot action; inject partial failure, stale beliefs, and sensor error; measure compensation and reconciliation.
9. **Future-capability sweeps.** Evaluate architecture across increasing model competence, verifier quality, attempt budgets, and interface access. Use oracle components where necessary to emulate future subsystems instead of freezing evaluation at today’s weakest model.

15.2 Indicators to track

Table 7: Leading indicators and disconfirming evidence.

Indicator	Supports the thesis if . . .	Weakens the strong thesis if . . .
Quality-adjusted capability cost	Fixed useful capabilities continue diffusing to cheap and local deployment	Costs and useful task reliability plateau across multiple architectures
Open-to-closed lag	Capability gaps repeatedly close through release, distillation, or independent invention	A persistent multi-year gap widens across economically important tasks
Agent task horizon	Reliable 50/80/99% horizons extend on non-gameable, real workflows	Long-horizon reliability remains flat despite better interfaces and verifiers
Human attention per outcome	Review minutes fall while defect and incident rates remain controlled	Generation rises but human verification grows proportionally
Production authorship	Agent-authored and agent-reviewed changes dominate validated deployment	Organizations revert because maintenance and security deteriorate
State portability	Switching models/providers preserves semantics, history, policy, and identity	State remains practically captive to vendors and application formats
Revenue mix	Seats and feature access decline relative to actions, outcomes, guarantees, and infrastructure	Seat-based application rent remains stable despite cheap reproduction
Semantic interfaces	Direct state/action contracts become normal for software and devices	GUI imitation remains equally effective and safer at comparable cost

Indicator	Supports the thesis if . .	Weakens the strong thesis if . .
Automated R&D	Independently verified agent discoveries materially improve deployed models, systems, or experiments	Proposals scale but validation yield remains negligible
Embodied cost/reliability	General manipulation expands beyond tightly structured environments	Physical tail risk and maintenance prevent broad economic deployment
Hardware openness	Open ISAs, chiplets, roots of trust, and automated EDA broaden implementation	Fabrication and platform control make open design operationally irrelevant

The strongest disconfirmation would be a multi-year joint plateau in model capability, quality-adjusted cost, open diffusion, semantic-interface gains, automated verification, and agent-authored production. One slow curve does not falsify the architecture; several stalled mechanisms would falsify the abundance case.

16 A Concrete End-State Vignette

A person asks to launch a multilingual educational service for a specific profession. No application is selected.

The system translates the goal into acceptance conditions, jurisdictional constraints, an expenditure envelope, and unanswered value choices. Research agents inspect public and licensed sources; a domain agent proposes curricula; a design agent creates multiple pedagogical structures; legal and safety validators flag claims; market agents run bounded experiments. A generated visual workspace appears only for decisions that benefit from comparison.

The accepted curriculum, contracts, identities, budgets, evidence, and outcome metrics become canonical state. Implementations for web, phone, television, accessibility devices, and partner portals are compiled as projections. A local model handles private personalization; expensive frontier cognition is invoked only where expected value justifies it. Code is generated, tested, attested, deployed, observed, and replaced. No vendor owns the project merely because its tool rendered a screen.

If physical kits are required, supply, payment, inventory, fabrication, shipment, and robotic handling become capability-bound transactions. Some operations are reversible, some compensatable, and some require explicit consent. Every consequential change produces a receipt. The person spends time on purpose, trade-offs, taste, stakeholder commitments, and surprising evidence—not on navigating folders and moving data among applications.

This vignette is not a user-interface concept. It is the consequence of changing which objects the computer considers fundamental.

17 Conclusion

The application-centric computer was a brilliant solution for human operators and scarce software production. It should not be mistaken for the final form of computing.

Frontier discovery, open diffusion, distillation, sparse activation, latent computation, persistent memory, automated research, specialized hardware, and embodied systems are changing at different rates. No single benchmark tells us their endpoint. Taken together, they justify designing for a regime in which cognitive attempts and implementation are abundant, even while energy, authority, validation, institutions, and contact with the physical world remain scarce.

The machine-native computer’s durable object is the verified semantic state transition. Its operating system is a governed fabric for state, cognition, capability, concurrency, execution, evidence, and learning. Applications become capability packages; files and screens become projections; code becomes a regenerable artifact; hardware becomes a heterogeneous placement fabric; models become pageable, substitutable workers.

Economically, this is not the end of paying for all computing. It is the end of implementation alone justifying durable rent. What remains billable is the right and ability to produce trusted consequence: current state, identity, compute, energy, live data, physical capacity, coordination, guarantees, and liability.

The central design test is severe: if models become far more capable, open, cheap, latent, self-improving, and embodied than they are today, does the architecture become less necessary? For a state-and-effect computer, the answer is no. Greater capability increases the value of a stable boundary between what intelligence can imagine and what an authorized system will allow reality to become.

A Relationship to Precursor Work

This paper synthesizes a project lineage rather than claiming that every component is new. Launchy exposed the cost of adapting a media product through fixed tools. MotionVector and its `docIR` showed that semantic authoring state can support deterministic rendering, multiple projections, extensible domain packs, and human-agent co-editing. SpacePilot reframed model deployment as a revisioned execution-path graph spanning artifact, runtime, hardware, placement, measured outcome, and evidence. The earlier Machine-Native Computer manuscript proposed canonical semantic state and a model-neutral machine-state interface.

The present contribution is the destination-first integration:

- current model and productivity limits are explicitly demoted from ceilings to launch conditions;
- support-aware semantic transactions become the narrow waist;
- model trajectories are incorporated as dynamic, routed, latent, and continually learning cognition;
- hardware is treated as an open, heterogeneous, attested placement fabric;
- digital and physical consequence share one effect model;
- the economic thesis is sharpened from software disappearance to implementation-rent collapse;
- scenarios and falsification tests replace linear extrapolation.

B Minimum Viable Machine-Native Substrate

A first research implementation need not replace an operating system. It can provide:

1. typed object IDs, revisions, provenance, and schema versions;
2. transaction proposals with read, support, and write sets;
3. capability leases scoped by object, operation, purpose, time, spend, and disclosure;
4. preconditions, invariants, expected effects, verification, and compensation;
5. a deterministic commit kernel and append-only receipt log;
6. adapters for `docIR`, Git, calendar, communication, and one physical simulator;
7. an execution graph that records exact model, artifact, runtime, hardware, and placement;
8. generated visual projections over the same state used by agents;
9. capability and interface benchmarks across model scales.

The first success criterion is not a beautiful assistant. It is two agents and one human safely producing concurrent, inspectable, model-substitutable changes to shared state with fewer unnecessary conflicts and clearer authority than the application-centric baseline.

References

- [1] METR. “Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity.” 10 July 2025. <https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/>
- [2] METR. “Uplift Update: Measuring AI’s Impact on Developer Productivity.” 24 February 2026. <https://metr.org/blog/2026-02-24-uplift-update/>

- [3] A. Ho et al. “Algorithmic Progress in Language Models.” arXiv:2403.05812, 2024. <https://arxiv.org/abs/2403.05812>
- [4] M. Gundlach et al. “The Price of Progress: Scaling Computation and Costs in AI.” arXiv:2511.23455, revised 2026. <https://arxiv.org/abs/2511.23455>
- [5] DeepSeek-AI. “DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.” arXiv:2501.12948, 2025. <https://arxiv.org/abs/2501.12948>
- [6] Qwen Team. “Qwen3 Technical Report.” arXiv:2505.09388, 2025. <https://arxiv.org/abs/2505.09388>
- [7] C. Snell et al. “Scaling LLM Test-Time Compute Optimally Can Be More Effective Than Scaling Model Parameters.” arXiv:2408.03314, 2024. <https://arxiv.org/abs/2408.03314>
- [8] S. Hao et al. “Training Large Language Models to Reason in a Continuous Latent Space.” arXiv:2412.06769, 2024. <https://arxiv.org/abs/2412.06769>
- [9] L. Barrault et al. “Large Concept Models: Language Modeling in a Sentence Representation Space.” arXiv:2412.08821, 2024. <https://arxiv.org/abs/2412.08821>
- [10] Z. Du et al. “Enabling Agents to Communicate Entirely in Latent Space.” arXiv:2511.09149, revised 2026. <https://arxiv.org/abs/2511.09149>
- [11] Y. Peng et al. “StateBridge: Training-Free Hidden-State Alignment for Latent Communication in LLM Multi-Agent Systems.” arXiv:2608.13317, 2026. <https://arxiv.org/abs/2608.13317>
- [12] S. Nie et al. “Large Language Diffusion Models.” arXiv:2502.09992, 2025. <https://arxiv.org/abs/2502.09992>
- [13] A. Behrouz et al. “Titans: Learning to Memorize at Test Time.” arXiv:2501.00663, 2024. <https://arxiv.org/abs/2501.00663>
- [14] A. Novikov et al. “AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery.” arXiv:2506.13131, 2025. <https://arxiv.org/abs/2506.13131>
- [15] J. Zhang et al. “Darwin Gödel Machine: Open-Ended Evolution of Self-Improving Agents.” arXiv:2505.22954, 2025. <https://arxiv.org/abs/2505.22954>
- [16] Y. Yamada et al. “The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search.” arXiv:2504.08066, 2025. <https://arxiv.org/abs/2504.08066>
- [17] J. Gottweis et al. “Accelerating Scientific Breakthroughs with an AI Co-Scientist.” arXiv:2502.18864, 2025. <https://arxiv.org/abs/2502.18864>
- [18] J. Gao et al. “UniLabOS: An AI-Native Operating System for Autonomous Laboratories.” arXiv:2512.21766, 2025. <https://arxiv.org/abs/2512.21766>
- [19] Bytecode Alliance. “WebAssembly System Interface.” Accessed August 2026. <https://wasi.dev/>
- [20] seL4 Foundation. “What Is seL4?” Accessed August 2026. <https://sel4.systems/About/whitepaper.html>
- [21] University of Cambridge. “CHERI: Capability Hardware Enhanced RISC Instructions.” Accessed August 2026. <https://www.cl.cam.ac.uk/research/security/ctsrd/cheri/>
- [22] M. Kleppmann et al. “Local-First Software: You Own Your Data, in Spite of the Cloud.” Onward! 2019. <https://www.inkandswitch.com/local-first/>
- [23] P. Bailis et al. “Coordination Avoidance in Database Systems.” arXiv:1402.2237, 2014. <https://arxiv.org/abs/1402.2237>
- [24] J. Hellerstein and P. Alvaro. “Keeping CALM: When Distributed Consistency Is Easy.” arXiv:1901.01930, 2019. <https://arxiv.org/abs/1901.01930>
- [25] H. Garcia-Molina and K. Salem. “Sagas.” ACM SIGMOD, 1987. <https://dl.acm.org/doi/10.1145/38713.38742>
- [26] Z. Li et al. “Agent-Semantic Interface Layer: Enabling Agents to Operate Software Through Semantic State and Actions.” arXiv:2608.26991, 2026. <https://arxiv.org/abs/2608.26991>
- [27] “CI4A: Component Interfaces for Agents.” arXiv:2601.14790, 2026. <https://arxiv.org/abs/2601.14790>
- [28] A2UI Working Group. “A2UI Specification.” Accessed August 2026. <https://a2ui.org/>
- [29] Google Research. “Generative UI: A Rich Custom Visual Interactive User Experience for Any Prompt.” 18 November 2025. <https://research.google/blog/generative-ui-a-rich-custom-visual-interactive-user-experience-for-any-prompt/>
- [30] K. Mei et al. “AIOS: LLM Agent Operating System.” arXiv:2403.16971, revised 2025. <https://arxiv.org/abs/2403.16971>
- [31] I. Shumailov et al. “AI Models Collapse When Trained on Recursively Generated Data.” Nature 631, 2024. <https://www.nature.com/articles/s41586-024-07566-y>

- [32] International Energy Agency. “Key Questions on Energy and AI: Executive Summary.” 16 April 2026. <https://www.iea.org/reports/key-questions-on-energy-and-ai/executive-summary>
- [33] Google Cloud. “TPU 8t and TPU 8i Technical Deep Dive.” 22 April 2026. <https://cloud.google.com/blog/products/compute/tpu-8t-and-tpu-8i-technical-deep-dive>
- [34] Micron. “High-Volume Production of HBM4.” 16 March 2026. <https://investors.micron.com/news/press-release/2026/Micron-in-High-Volume-Production-of-HBM4-Designed-for-NVIDIA-Vera-Rubin-PCIe-Gen6-SSD-and-SOC-AMM2-03-16-2026/default.aspx>
- [35] UCIE Consortium. “UCIE Specifications.” Accessed August 2026. <https://www.uciexpress.org/specifications>
- [36] TSMC. “TSMC Unveils Next-Generation A14 Process at North America Technology Symposium.” 24 April 2025. <https://pr.tsmc.com/english/news/3228>
- [37] M. Le Gallo et al. “A 64-Core Mixed-Signal In-Memory Compute Chip Based on Phase-Change Memory for Deep Neural Network Inference.” *Nature Electronics*, 2023. <https://research.ibm.com/publications/a-64-core-mixed-signal-in-memory-compute-chip-based-on-phase-change-memory-for-deep-neural-network-inference>
- [38] RISC-V International. “The Technical Backbone of RISC-V.” Accessed August 2026. <https://riscv.org/>
- [39] OpenROAD Project. “Open-Source Autonomous Chip Design Toolchain.” Accessed August 2026. <https://theopenroadproject.org/>
- [40] lowRISC. “OpenTitan: Open Source Silicon Root of Trust.” Accessed August 2026. <https://opentitan.org/>
- [41] Google DeepMind. “Gemini Robotics Brings AI into the Physical World.” 12 March 2025. <https://deepmind.google/blog/gemini-robotics-brings-ai-into-the-physical-world/>
- [42] Physical Intelligence. “ π_0 : Our First Generalist Policy.” 31 October 2024. <https://www.physicalintelligence.com/p/blog/pi0>
- [43] NVIDIA. “Cosmos World Foundation Model Platform for Physical AI.” 6 January 2025. https://research.nvidia.com/publication/2025-01_cosmos-world-foundation-model-platform-physical-ai
- [44] NVIDIA. “Introducing Jetson Thor for Physical AI.” 25 August 2025. <https://developer.nvidia.com/blog/introducing-nvidia-jetson-thor-the-ultimate-platform-for-physical-ai/>
- [45] K. Cruse et al. “Scaling Laws of Scientific Discovery with AI and Robot Scientists.” arXiv:2503.22444, 2025. <https://arxiv.org/abs/2503.22444>
- [46] Intercom. “Building Outcome-Based Pricing for Fin for Sales.” 8 May 2026. <https://www.intercom.com/blog/building-outcome-based-pricing-for-fin-for-sales/>
- [47] Salesforce. “Agentforce Pricing.” Accessed August 2026. <https://www.salesforce.com/ap/agentforce/pricing/>
- [48] Model Context Protocol. “Specification, 28 July 2026.” 2026. <https://modelcontextprotocol.io/specification/2026-07-28>
- [49] A2A Project. “Agent2Agent Protocol Specification 1.0.” Accessed August 2026. <https://a2a-protocol.org/latest/specification/>
- [50] W3C Web Machine Learning Community Group. “WebMCP.” Accessed August 2026. <https://github.com/webmachinelearning/webmcp>
- [51] Ink & Switch. “Cambria: Translate Your Data with Lenses.” 6 October 2020. <https://www.inkandswitch.com/cambria/>